

Zomato Customer Analytics Case Study

Zomato is one of India's largest food delivery platforms, connecting millions of customers with restaurants across multiple cities.

Over the last few months, the leadership team has observed that while the number of registered customers continues to grow, overall business performance is not improving at the expected rate. Revenue growth has become inconsistent, customer retention appears to be declining, and certain restaurant partners are underperforming.

The management team has hired a Data Analyst to investigate the business and answer critical questions that can help improve growth, customer retention, and operational efficiency.

As a Data Analyst, your objective is to analyze customer, restaurant, and order data to uncover actionable insights and provide data-driven recommendations.

- Revenue Analysis,
- Customer Analysis
- Restaurant Performance
- Coupon Analysis
- Cancellation & Refund Analysis
- Customer Churn Analysis

Revenue Analysis

1. What is the total revenue?

SQL Query:

```
SELECT SUM(order_amount) AS total_revenue  
FROM Orders  
WHERE order_status = 'Delivered';
```

Output:

100 %

Results Messages

	total_revenue
1	26859391.4037476

2. What is the monthly revenue trend?

SQL Query:

```
SELECT datetrunc(month,order_timestamp) AS month, sum(order_amount) AS total_revenue
FROM Orders
WHERE order_status = 'Delivered'
GROUP BY datetrunc(month,order_timestamp)
ORDER BY month asc;
```

Output:

100 %

Results Messages

	month	total_revenue
1	2024-01-01	993756.919570923
2	2024-02-01	914237.788696289
3	2024-03-01	976562.869888306
4	2024-04-01	975680.631530762
5	2024-05-01	959287.189743042
6	2024-06-01	975701.109466553
7	2024-07-01	930763.57901001
8	2024-08-01	984926.560058594
9	2024-09-01	926511.55090332
10	2024-10-01	1006185.46810913
11	2024-11-01	969876.559280396
12	2024-12-01	981302.95954895
13	2025-01-01	1013576.52134705
14	2025-02-01	838330.509490967
15	2025-03-01	1000685.05943298

Query executed successfully

3. Which city contributes the highest revenue?

SQL Query:

```
SELECT a.City, sum(order_amount) AS total_revenue
FROM Customers a
JOIN Orders o
ON a.customer_id = o.customer_id
WHERE order_status = 'Delivered'
GROUP BY a.city
ORDER BY SUM(order_amount) desc;
```

Output:

Results		Messages
	City	total_revenue
1	Noida	3604414.9690094
2	Pune	3539648.36965942
3	Gurugram	3358890.34896851
4	Hyderabad	3305347.99267578
5	Bengaluru	3294090.17964172
6	Delhi	3292014.60681152
7	Mumbai	3242934.96826172
8	Chennai	3217081.56872559

4. Which payment mode generates the most revenue?

SQL Query:

```
SELECT payment_mode, SUM(order_amount) AS total_revenue
FROM Orders
WHERE order_status = 'Delivered'
GROUP BY payment_mode
ORDER BY SUM(order_amount) desc;
```

Output:

Results Messages		
	payment_mode	total_revenue
1	UPI	13433746.6039886
2	Wallet	4514679.88328552
3	Card	4474070.29698181
4	Cash	4436894.61949158

5. What is the Average Order Value (AOV)?

SQL Query:

```
SELECT SUM(order_amount)/count(distinct order_id) AS average_order_value
FROM Orders
WHERE order_status = 'Delivered';
```

Output:

Results Messages	
	average_order_value
1	900.204156039399

Customer Analysis

1. Who are the top 20 customers by revenue?

SQL Query:

```
SELECT TOP 20
  customer_id,
  SUM(order_amount - discount_amount + delivery_fee) AS revenue,
  COUNT(order_id) AS delivered_orders
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY revenue DESC;
```

Output:

Results		Messages	
	customer_id	revenue	delivered_orders
1	C00381	15932.9900741577	16
2	C00701	15683.5300312042	17
3	C03890	15464.4099960327	15
4	C01409	14773.0898399353	16
5	C01659	14433.2698516846	14
6	C02596	14040.830039978	12
7	C01957	13936.4500579834	13
8	C01795	13853.6899871826	12
9	C01729	13830.399848938	13
10	C02739	13713.0701141357	13
11	C02235	13567.3499336243	15
12	C04756	13305.5402526855	13
13	C01468	13297.9000587463	13
14	C04598	13291.9299621582	14
15	C03054	13251.3399276733	13

Query executed successfully

2. What percentage of revenue comes from top customers?

SQL Query:

```

WITH customer_revenue AS (
  SELECT
    customer_id,
    SUM(order_amount - discount_amount + delivery_fee) AS revenue
  FROM orders
  WHERE order_status = 'Delivered'
  GROUP BY customer_id
),
ranked_customers AS (
  SELECT
    customer_id,
    revenue,
    ROW_NUMBER() OVER (ORDER BY revenue DESC) AS customer_rank
  FROM customer_revenue
)
SELECT
  SUM(CASE WHEN customer_rank <= 20 THEN revenue ELSE 0 END) AS top_20_revenue,
  SUM(revenue) AS total_revenue,

```

```

ROUND(
    100.0 * SUM(CASE WHEN customer_rank <= 20 THEN revenue ELSE 0 END)
    / SUM(revenue),
    2
) AS top_20_revenue_percentage
FROM ranked_customers;

```

Output:

100 %			
Results Messages			
	top_20_revenue	total_revenue	top_20_revenue_percentage
1	278248.750200272	26965797.1539078	1.03

3. Which acquisition channel brings the highest-value customers?

SQL Query:

```

SELECT
    c.Acquisition_channel,
    COUNT(DISTINCT o.customer_id) AS customers,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS total_revenue,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee)
    / COUNT(DISTINCT o.customer_id) AS revenue_per_customer
FROM orders o
JOIN Customers c
    ON o.customer_id = c.Customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.Acquisition_channel
ORDER BY revenue_per_customer DESC;

```

Output:

Results Messages				
	Acquisition_channel	customers	total_revenue	revenue_per_customer
1	WhatsApp Campaign	989	5432472.02135658	5492.89385374781
2	Google Ads	990	5406384.09809303	5460.99403847781
3	Organic	987	5350063.55837631	5420.53045428198
4	Referral	1000	5364781.18751335	5364.78118751335
5	Instagram Ads	1024	5406991.48857307	5280.26512555964

4. How many repeat customers do we have?

SQL Query:

```
WITH frst AS (  
    SELECT  
        customer_id,  
        MIN(order_timestamp) AS first_time  
    FROM Orders  
    GROUP BY customer_id  
)  
SELECT  
    DATETRUNC(month, a.order_timestamp) AS Month,  
    COUNT(DISTINCT a.customer_id) AS repeated_cust  
FROM Orders a  
JOIN frst b  
    ON a.customer_id = b.customer_id  
    AND DATETRUNC(month, a.order_timestamp) <> DATETRUNC(month, b.first_time)  
GROUP BY DATETRUNC(month, a.order_timestamp)  
ORDER BY Month;
```

Output:

Results		Messages
	Month	repeated_cust
1	2024-02-01	432
2	2024-03-01	773
3	2024-04-01	1009
4	2024-05-01	1181
5	2024-06-01	1254
6	2024-07-01	1283
7	2024-08-01	1381
8	2024-09-01	1416
9	2024-10-01	1497
10	2024-11-01	1448
11	2024-12-01	1473
12	2025-01-01	1566
13	2025-02-01	1385
14	2025-03-01	1473
15	2025-04-01	1449

Restaurant Performance

1. Which restaurants generate the highest revenue?

SQL Query:

```
SELECT TOP 10
    r.restaurant_id,
    r.restaurant_name,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS revenue
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY revenue DESC;
```

Output:

	restaurant_id	restaurant_name	revenue
1	159.00	KFC	166589.759836197
2	72.00	Biryani By Kilo	160277.99036026
3	149.00	Barbeque Nation	159827.850112915
4	193.00	McDonald's	158718.8396492
5	116.00	Haldiram's	158325.139640808
6	40.00	Behrouz Biryani	158298.460926056
7	58.00	KFC	158069.510389328
8	199.00	Burger King	157824.969747543
9	192.00	Haldiram's	156491.850166321
10	62.00	Subway	155758.470384598

2. Which restaurants receive the most orders?

SQL Query:

```
SELECT TOP 10
    r.restaurant_id,
    r.restaurant_name,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN restaurants r
```

```

ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY total_orders DESC;

```

Output:

	restaurant_id	restaurant_name	total_orders
1	159.00	KFC	185
2	116.00	Haldiram's	181
3	72.00	Biryani By Kilo	179
4	193.00	McDonald's	177
5	133.00	Burger King	175
6	82.00	Chaayos	174
7	58.00	KFC	174
8	149.00	Barbeque Nation	174
9	62.00	Subway	173
10	111.00	Biryani By Kilo	173

3. Which cuisines are most popular?

SQL Query:

```

SELECT
    r.cuisine,
    COUNT(o.order_id) AS total_orders,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS revenue
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.cuisine
ORDER BY total_orders DESC;

```

Output:

Results		Messages	
	cuisine	total_orders	revenue
1	North Indian	5869	5254133.56032181
2	Pizza	5706	5161393.74858856
3	Fast Food	5378	4872601.84784889
4	Cafe	4536	4117375.39120674
5	Healthy	4489	4076357.82613564
6	Biryani	3859	3483934.77980614

4. Do highly-rated restaurants generate more revenue?

SQL Query:

```
SELECT
    COUNT(DISTINCT r.restaurant_id) AS restaurants,
    r.avg_rating,
    AVG(CAST(x.revenue AS DECIMAL(18,2))) AS avg_revenue
FROM restaurants r
JOIN (
    SELECT
        restaurant_id,
        SUM(order_amount - discount_amount + delivery_fee) AS revenue
    FROM orders
    WHERE order_status = 'Delivered'
    GROUP BY restaurant_id
) x
ON r.restaurant_id = x.restaurant_id
GROUP BY r.avg_rating
ORDER BY r.avg_rating desc;
```

Output:

	restaurants	avg_rating	avg_revenue
1	5	5	138946.402000
2	13	4.90000009536743	140164.538461
3	14	4.80000019073486	135402.360714
4	10	4.69999980926514	131067.615000
5	18	4.59999990463257	136729.352777
6	17	4.5	132541.000588
7	13	4.40000009536743	135398.938461
8	12	4.30000019073486	128300.195833
9	17	4.19999980926514	134769.884705
10	13	4.09999990463257	129845.656923
11	17	4	137923.265882
12	7	3.90000009536743	133073.395714
13	15	3.79999995231628	133886.484000
14	13	3.70000004768372	138270.165384
15	9	3.59999990463257	135003.413333

5. Last 5 Restaurant

SQL Query:

– Based on Average ratings

```
SELECT TOP 5
    restaurant_id,
    restaurant_name,
    avg_rating
FROM restaurants
ORDER BY avg_rating ASC;
```

– Based on revenue

```
SELECT TOP 5
    r.restaurant_id,
    r.restaurant_name,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS revenue,
    r.avg_rating
FROM restaurants r
join orders o
on r.restaurant_id = o.restaurant_id
group by r.restaurant_id, r.restaurant_name, r.avg_rating
ORDER BY r.avg_rating asc;
```

Output:

Results		Messages	
	restaurant_id	restaurant_name	avg_rating
1	26.00	Haldiram's	3.5
2	47.00	Starbucks	3.5
3	65.00	Burger King	3.5
4	89.00	Starbucks	3.5
5	111.00	Biryani By Kilo	3.5

	restaurant_id	restaurant_name	revenue	avg_rating
1	26.00	Haldiram's	228833.899934769	3.5
2	47.00	Starbucks	212167.410167694	3.5
3	65.00	Burger King	223368.01076889	3.5
4	89.00	Starbucks	233790.080192566	3.5
5	111.00	Biryani By Kilo	245838.890274048	3.5

Coupon Analysis

1. What percentage of orders use coupons?

```
SELECT
  COUNT(*) AS total_delivered_orders,
  SUM(CASE WHEN discount_amount > 0 THEN 1 ELSE 0 END) AS coupon_orders,
  ROUND(
    100.0 * SUM(CASE WHEN discount_amount > 0 THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS coupon_usage_percentage
FROM orders
WHERE order_status = 'Delivered';
```

Output:

	total_delivered_orders	coupon_orders	coupon_usage_percentage
1	29837	12030	40.3200000000000

2. Do coupon users spend more than non-coupon users?

SQL Query:

```
SELECT
CASE
    WHEN discount_amount > 0 THEN 'Coupon User'
    ELSE 'Non-Coupon User'
END AS customer_type,
COUNT(*) AS orders,
ROUND(AVG(order_amount), 2) AS avg_order_value,
ROUND(
    AVG(order_amount - discount_amount + delivery_fee),
    2
) AS avg_realized_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY
CASE
    WHEN discount_amount > 0 THEN 'Coupon User'
    ELSE 'Non-Coupon User'
END;
```

Output:

	customer_type	orders	avg_order_value	avg_realized_revenue
1	Coupon User	12030	904.59	854.14
2	Non-Coupon User	17807	897.24	937.3

3. Which city uses the most coupons?

SQL Query:

```
SELECT
    c.City,
    SUM(CASE WHEN o.discount_amount > 0 THEN 1 ELSE 0 END) AS coupon_orders
FROM orders o
JOIN customers c
    ON o.customer_id = c.Customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.City
ORDER BY coupon_orders DESC;
```

Output:

	City	coupon_orders
1	Noida	1600
2	Pune	1598
3	Delhi	1507
4	Gurugram	1483
5	Bengaluru	1482
6	Hyderabad	1474
7	Chennai	1467
8	Mumbai	1418

4. Are coupons helping customer retention?

SQL Query:

```
WITH customer_orders AS (
    SELECT
        customer_id,
        COUNT(order_id) AS total_orders,
        SUM(CASE WHEN discount_amount > 0 THEN 1 ELSE 0 END) AS coupon_orders
    FROM orders
    WHERE order_status = 'Delivered'
    GROUP BY customer_id
),
customer_segments AS (
    SELECT
        customer_id,
```

```

total_orders,
CASE
  WHEN coupon_orders > 0 THEN 'Coupon User'
  ELSE 'Non-Coupon User'
END AS customer_type,
CASE
  WHEN total_orders > 1 THEN 1
  ELSE 0
END AS is_repeat
FROM customer_orders
)
SELECT
  customer_type,
  COUNT(*) AS customers,
  SUM(is_repeat) AS repeat_customers,
  ROUND(
    100.0 * SUM(is_repeat) / COUNT(*),
    2
  ) AS repeat_rate
FROM customer_segments
GROUP BY customer_type;

```

Output:

	customer_type	customers	repeat_customers	repeat_rate
1	Coupon User	4559	4530	99.36000000000000
2	Non-Coupon User	432	385	89.12000000000000

Cancellation & Refund Analysis

1. What is the cancellation rate?

SQL Query:

```

SELECT
  COUNT(*) AS total_orders,
  SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
  ROUND(
    100.0 * SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS cancellation_rate

```

FROM orders;

Output:

Results		Messages	
	total_orders	cancelled_orders	cancellation_rate
1	50000	10095	20.1900000000000

2. What is the refund rate?

SQL Query:

```
SELECT
    COUNT(*) AS total_orders,
    SUM(CASE WHEN order_status = 'Refunded' THEN 1 ELSE 0 END) AS refunded_orders,
    ROUND(
        100.0 * SUM(CASE WHEN order_status = 'Refunded' THEN 1 ELSE 0 END)
        / COUNT(*),
        2
    ) AS refund_rate
FROM orders;
```

Output:

Results		Messages	
	total_orders	refunded_orders	refund_rate
1	50000	10068	20.1400000000000

3. How much revenue is lost due to cancellations?

SQL Query:

```
SELECT
    SUM(
        order_amount
        - discount_amount
        + delivery_fee
    ) AS lost_revenue
FROM orders
WHERE order_status = 'Cancelled';
```

Output:

Results		Messages
	lost_revenue	
1	9094421.69070625	

4. Which restaurants have the highest cancellation rate?

SQL Query:

```
SELECT TOP 10
  r.restaurant_id,
  r.restaurant_name,
  COUNT(o.order_id) AS total_orders,
  SUM(
    CASE
      WHEN o.order_status = 'Cancelled' THEN 1
      ELSE 0
    END
  ) AS cancelled_orders,
  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN o.order_status = 'Cancelled' THEN 1
        ELSE 0
      END
    ) / COUNT(o.order_id),
    2
  ) AS cancellation_rate
FROM orders o
JOIN restaurants r
  ON o.restaurant_id = r.restaurant_id
GROUP BY
  r.restaurant_id,
  r.restaurant_name
ORDER BY cancellation_rate DESC;
```

Output:

	restaurant_id	restaurant_name	total_orders	cancelled_orders	cancellation_rate
1	194.00	Haldiram's	245	67	27.3500000000000
2	185.00	Barbeque Nation	251	66	26.2900000000000
3	120.00	Biryani By Kilo	222	58	26.1300000000000
4	46.00	KFC	243	63	25.9300000000000
5	64.00	Chaayos	263	66	25.1000000000000
6	155.00	McDonald's	267	67	25.0900000000000
7	49.00	Barbeque Nation	240	60	25.0000000000000
8	21.00	Barbeque Nation	241	60	24.9000000000000
9	22.00	Behrouz Biryani	245	61	24.9000000000000
10	99.00	Chaayos	267	66	24.7200000000000

Customer Churn Analysis

1. How many customers have churned?

SQL Query:

```
WITH last_order AS (
    SELECT
        customer_id,
        MAX(order_timestamp) AS last_order_date
    FROM orders
    WHERE order_status = 'Delivered'
    GROUP BY customer_id
)
SELECT
    COUNT(*) AS churned_customers
FROM customers c
LEFT JOIN last_order l
    ON c.Customer_id = l.customer_id
WHERE l.last_order_date < DATEADD(DAY, -90, '2026-04-30')
    OR l.last_order_date IS NULL;
```

Output:

	churned_customers
1	2672

2. What is the churn rate?

SQL Query:

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
)  
SELECT  
    COUNT(*) AS total_customers,  
    SUM(  
        CASE  
            WHEN l.last_order_date < DATEADD(DAY, -90, '2026-04-30')  
            OR l.last_order_date IS NULL  
            THEN 1 ELSE 0  
        END  
    ) AS churned_customers,  
    ROUND(  
        100.0 * SUM(  
            CASE  
                WHEN l.last_order_date < DATEADD(DAY, -90, '2026-04-30')  
                OR l.last_order_date IS NULL  
                THEN 1 ELSE 0  
            END  
        ) / COUNT(*),  
        2  
    ) AS churn_rate  
FROM customers c  
LEFT JOIN last_order l  
    ON c.Customer_id = l.customer_id;
```

Output:

 Results  Messages			
	total_customers	churned_customers	churn_rate
1	4999	2672	53.450000000000

3. Which city has the highest churn?

SQL Query:

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
)  
customer_churn AS (  
    SELECT  
        c.Customer_id,  
        c.City,  
        CASE  
            WHEN l.last_order_date < DATEADD(DAY, -90, '2026-04-30')  
            OR l.last_order_date IS NULL  
            THEN 1 ELSE 0  
        END AS churned  
    FROM customers c  
    LEFT JOIN last_order l  
        ON c.Customer_id = l.customer_id  
)  
SELECT  
    City,  
    COUNT(*) AS total_customers,  
    SUM(churned) AS churned_customers,  
    ROUND(100.0 * SUM(churned) / COUNT(*), 2) AS churn_rate  
FROM customer_churn  
GROUP BY City  
ORDER BY churn_rate DESC;
```

Output:

Results		Messages		
	City	total_customers	churned_customers	churn_rate
1	Noida	666	367	55.11000000000000
2	Bengaluru	619	340	54.93000000000000
3	Chennai	618	338	54.69000000000000
4	Pune	644	351	54.50000000000000
5	Delhi	612	323	52.78000000000000
6	Gurugram	613	321	52.37000000000000
7	Mumbai	612	317	51.80000000000000
8	Hyderabad	615	315	51.22000000000000

4. How much revenue is lost due to churn?

SQL Query:

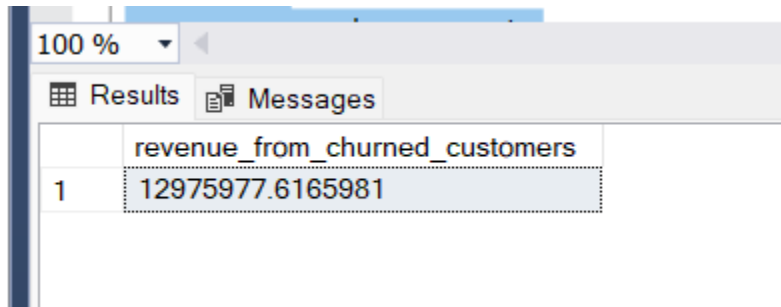
```

WITH last_order AS (
    SELECT
        customer_id,
        MAX(order_timestamp) AS last_order_date
    FROM orders
    WHERE order_status = 'Delivered'
    GROUP BY customer_id
),
churned_customers AS (
    SELECT
        c.Customer_id
    FROM customers c
    LEFT JOIN last_order l
        ON c.Customer_id = l.customer_id
    WHERE l.last_order_date < DATEADD(DAY, -90, '2026-04-30')
        OR l.last_order_date IS NULL
)
SELECT
    SUM(
        o.order_amount
        - o.discount_amount
        + o.delivery_fee
    ) AS revenue_from_churned_customers
FROM orders o
JOIN churned_customers c

```

ON o.customer_id = c.Customer_id
WHERE o.order_status = 'Delivered';

Output:



The screenshot shows a SQL query results window. At the top, there is a zoom level dropdown set to '100 %'. Below it are two tabs: 'Results' (active) and 'Messages'. The 'Results' tab displays a table with one column, 'revenue_from_churned_customers', and one row with the value '12975977.6165981'. The row is highlighted with a blue background.

	revenue_from_churned_customers
1	12975977.6165981

5. Who are the high-value churned customers?

SQL Query:

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
)  
churned_customers AS (  
    SELECT  
        c.Customer_id,  
        c.Customer_name,  
        c.City,  
        l.last_order_date  
    FROM customers c  
    LEFT JOIN last_order l  
        ON c.Customer_id = l.customer_id  
    WHERE l.last_order_date < DATEADD(DAY, -90, '2026-04-30')  
        OR l.last_order_date IS NULL  
)  
customer_revenue AS (  
    SELECT  
        o.customer_id,  
        SUM(  
            o.order_amount  
            - o.discount_amount  
            + o.delivery_fee
```

```

        ) AS lost_revenue,
        COUNT(o.order_id) AS delivered_orders
FROM orders o
WHERE o.order_status = 'Delivered'
GROUP BY o.customer_id
)
SELECT TOP 10
    c.Customer_id,
    c.Customer_name,
    c.City,
    r.lost_revenue,
    r.delivered_orders,
    c.last_order_date
FROM churned_customers c
JOIN customer_revenue r
    ON c.Customer_id = r.customer_id
ORDER BY r.lost_revenue DESC;

```

Output:

	Customer_id	Customer_name	City	lost_revenue	delivered_orders	last_order_date
1	C03054	Rahul Sharma	Noida	13251.3399276733	13	2026-01-22
2	C02364	Neha Singh	Noida	13163.420249939	12	2026-01-16
3	C04768	Neha Singh	Pune	12768.2999839783	13	2025-11-25
4	C01535	Rohan Agarwal	Pune	12484.5099334717	14	2026-01-28
5	C03220	Priya Verma	Chennai	12385.7900314331	14	2025-08-19
6	C00714	Ananya Mishra	Bengaluru	12378.669921875	14	2026-01-22
7	C04595	Priya Verma	Mumbai	12335.0100021362	10	2025-12-27
8	C03961	Neha Singh	Noida	12281.8001937866	10	2026-01-17
9	C01029	Amit Kumar	Mumbai	12026.5899581909	11	2026-01-21
10	C02582	Priya Verma	Delhi	11981.8800048828	11	2026-01-09